CHAPTER 2 THEORETICAL BACKGROUND

This chapter presents the components, training methods, and evaluation measures used in CNN-based road damage classification. The discussion considers single-label multiclass classification: each image is assigned to exactly one damage class. It also introduces representative CNN architectures, transfer learning, and data augmentation.

2.2. Convolutional Neural Networks

2.2.1. Overview and Basic Architecture of CNNs

A convolutional neural network (CNN) processes grid-structured data such as images by learning local patterns and combining them across successive layers. Local connectivity restricts each filter to a small region, while parameter sharing applies the same filter at different spatial positions. These properties reduce the number of parameters relative to a fully connected transformation of the same image [1, Ch. 9].

A typical CNN classifier consists of convolutional layers, nonlinear activation functions, optional normalization and pooling layers, and a classification head. Earlier layers can represent edges and textures, while later layers combine them into features useful for classification. Convolution provides translation equivariance under appropriate stride and boundary conditions: shifting the input shifts its feature responses. It does not by itself guarantee that the final prediction is invariant to every translation [1, Ch. 9].

2.2.2. Convolutional Layer

An input mini-batch is represented by $X \in \mathbb{R}^{B\times C_{\text{in}}\times H\times W}$, where B is the number of images, C_{in} is the number of input channels, and H, W are the image height and width. A channel is one two-dimensional component of the tensor. An RGB image has three channels; in a hidden layer, channels contain learned responses rather than raw color values. A feature map is the spatial array of responses associated with one output channel [2, Sec. 7.4].

In a standard convolutional layer with no grouping, each output filter spans all input channels. For kernel weights $K \in \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}\times k_h\times k_w}$, the pre-activation output is [2, Secs. 7.2–7.4; 3]:

$$Z_{n,o,i,j} = b_o + \sum_{c=0}^{C_{\text{in}}-1}\sum_{u=0}^{k_h-1}\sum_{v=0}^{k_w-1} K_{o,c,u,v}\ \tilde{X}_{n,c,is_h+u,js_w+v}$$

Here, $\tilde{X}$ is the zero-padded input; n, o, c index images, output channels, and input channels; i, j index output positions; and u, v index kernel positions. The strides s_h, s_w control the filter displacement. The formula assumes unit dilation. Deep learning libraries call this operation convolution, although it is mathematically cross-correlation because the kernel is not reversed [3].

$$H_{\text{out}} = \lfloor\frac{H + 2p_h - k_h}{s_h}\rfloor + 1, \quad W_{\text{out}} = \lfloor\frac{W + 2p_w - k_w}{s_w}\rfloor + 1$$

The padding amounts are p_h, p_w. Each of the C_{out} filters produces one feature map by summing its responses over all input channels. For example, 16 filters of size 3×3 applied to an RGB image use weights of shape $16 \times 3 \times 3 \times 3$ and produce 16 output channels. With stride 1 and padding 1, the spatial dimensions are preserved [2, Sec. 7.4; 3].

2.2.3. Activation Functions

Activation functions introduce nonlinearity into the network. Without nonlinear operations, a sequence of affine layers is equivalent to a single affine transformation. Common elementwise activation functions are [1, Sec. 6.3]:

$$\mathrm{Re\,LU}(z) = \max(0, z), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

ReLU is inexpensive and has derivative 1 for positive inputs and 0 for negative inputs. It avoids saturation on the positive side, although units that remain inactive can receive zero gradients. Sigmoid maps values to $(0, 1)$, and tanh maps them to $(-1,1)$; both can produce small gradients when the magnitude of their input is large. The project uses ReLU in its hidden layers [12].

2.2.4. Pooling Layer

Pooling summarizes a spatial neighborhood within each channel. It usually reduces height and width while preserving the number of channels, lowering the size of subsequent feature tensors. For a pooling region $\mathcal{R}_{i,j}$, max pooling and average pooling are defined as [1, Sec. 9.3]:

$$P^{\max}_{n,c,i,j} = \max_{(u,v)\in\mathcal{R}_{i,j}} A_{n,c,u,v}$$

$$P^{\mathrm{avg}}_{n,c,i,j} = \frac{1}{|\mathcal{R}_{i,j}|} \sum_{(u,v)\in\mathcal{R}_{i,j}} A_{n,c,u,v}$$

Max pooling retains the strongest response in a region, whereas average pooling summarizes all responses in that region. Pooling can reduce sensitivity to small local shifts, but it also discards spatial detail. Global average pooling reduces each entire feature map to one value; adaptive average pooling can instead produce a specified spatial grid [2, Sec. 7.5].

2.2.5. Fully Connected Layer and Softmax

After feature extraction, flattening or pooling converts feature maps into a vector $\mathbf{a}$. A fully connected layer combines these features through an affine transformation. The final layer produces one logit for each of the M classes [1, Ch. 6]:

$$\mathbf{z} = W_{\mathrm{fc}}\mathbf{a} + \mathbf{b}_{\mathrm{fc}}, \quad p_k = \frac{\exp(z_k)}{\sum_{j=1}^{M} \exp(z_j)}, \quad \hat{y} = \mathrm{argmax}_k p_k$$

Softmax converts logits into probabilities that sum to one. It is appropriate here because the classes are mutually exclusive. The project returns logits directly and passes them to PyTorch CrossEntropyLoss, which incorporates log-softmax internally; an explicit softmax layer is unnecessary during training [12; 13].

2.2.6. Batch Normalization

Batch normalization (BN) normalizes activations separately for each channel. For $X \in \mathbb{R}^{B\times C\times H\times W}$, training statistics are computed over the batch and all spatial positions, not across channels [4, Algorithm 1 and Sec. 3.2]:

$$\mu_c = \frac{1}{BHW}\sum_{n=1}^{B}\sum_{h=1}^{H}\sum_{w=1}^{W} X_{n,c,h,w}$$

$$\sigma_c^2 = \frac{1}{BHW}\sum_{n=1}^{B}\sum_{h=1}^{H}\sum_{w=1}^{W} (X_{n,c,h,w} - \mu_c)^2$$

$$Y_{n,c,h,w} = \gamma_c \frac{X_{n,c,h,w} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c$$

Each channel has a learned scale γ_c and shift β_c ; $\epsilon > 0$ ensures numerical stability. During inference, stored running statistics replace the current batch statistics. The project uses Conv–BN–ReLU blocks. This normalization supports optimization but does not decorrelate channels or perform full whitening [4; 12].

2.3. CNN Training and Optimization

2.3.1. Forward Propagation and Loss Function

Forward propagation transforms an input mini-batch through the network to obtain logits and class probabilities. For B examples and M classes, the mean categorical cross-entropy loss is [1, Sec. 6.2; 13]:

$$\mathcal{L}_{\mathcal{B}}(\theta) = -\frac{1}{B}\sum_{n=1}^{B}\sum_{k=1}^{M} y_{n,k}\log p_{n,k}(\theta)$$

The one-hot target $y_{n,k}$ equals 1 for the correct class and 0 otherwise; $p_{n,k}$ is the predicted probability, and θ contains the trainable parameters. Thus, each example contributes the negative logarithm of the probability assigned to its correct class. Minimizing this loss encourages larger probabilities for the observed labels. Integer class labels can be used directly in the implementation without explicitly creating one-hot vectors [13].

2.3.2. Backpropagation and Mini-Batch Gradient Descent

Backpropagation applies the chain rule from the loss toward earlier layers. It computes gradients; the optimizer then uses those gradients to update parameters. For the mean softmax cross-entropy loss above, the gradient with respect to each output logit is:

$$\frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial z_{n,k}} = \frac{p_{n,k} - y_{n,k}}{B}$$

To illustrate the backward pass of a convolution, consider the cross-correlation in Section 2.2.2 with unit stride, unit dilation, and no padding. Let $\delta_{n,o,i,j} = \partial \mathcal{L}_{\mathcal{B}} / \partial Z_{n,o,i,j}$ be the gradient arriving at its pre-activation output. Applying the chain rule gives:

$$\frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial K_{o,c,u,v}} = \sum\nolimits_{n,i,j} \delta_{n,o,i,j}\, X_{n,c,i+u,j+v}, \quad \frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial b_o} = \sum\nolimits_{n,i,j} \delta_{n,o,i,j}$$

$$\frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial X_{n,c,h,w}} = \sum\nolimits_{o} \sum\nolimits_{u} \sum\nolimits_{v} K_{o,c,u,v}\ \delta_{n,o,h-u,w-v}$$

The sums cover valid batch, spatial, and kernel indices; in the input-gradient formula, δ is zero outside its valid output range. The kernel gradient accumulates contributions from every location where the shared weight was used. The input gradient carries the error to earlier layers. These equations specialize the chain rule to the stated convolution; the factor $1/B$ is already included in δ and must not be applied again [1, Sec. 6.5 and Ch. 9; 2, Sec. 5.3].

For a ReLU applied directly to Z, the local backward rule is:

$$\delta = \frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial A} \odot \mathbf{1}[Z > 0], \quad A = \mathrm{Re\,LU}(Z)$$

Here, $\odot$ denotes elementwise multiplication. In Conv–BN–ReLU blocks, the gradient must also pass through BN before reaching the convolution. Max pooling routes the gradient to the selected maximum in each window; overlapping windows contribute additively [2, Sec. 5.3; 4].

Mini-batch training uses a small batch at each update rather than computing a full-dataset gradient. The basic mini-batch SGD update is:

$$g_t = \nabla_\theta \mathcal{L}_{\mathcal{B}_t}(\theta_t), \quad \theta_{t+1} = \theta_t - \eta_t g_t$$

The learning rate η_t controls the step size. The project uses shuffled mini-batches and applies Adam to their gradients [12].

2.3.3. Optimization Algorithms

Adam uses the mini-batch gradient g_t to update each parameter through running moment estimates. All vector products, squares, square roots, and divisions below are elementwise [5].

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad m_0 = 0$$

The first moment m_t smooths the gradient direction; β_1 controls how much past information is retained.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad v_0 = 0$$

The second raw moment v_t tracks squared gradient magnitudes. It is not a centered variance. The decay factor β_2 controls its memory.

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Bias correction compensates for initializing both moment estimates at zero.

$$\theta_{t+1} = \theta_t - \eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

The corrected first moment determines the update direction, while the denominator adapts its scale for each parameter. The learning rate η_t sets the overall step size, and $\epsilon > 0$ stabilizes division. When an L2 penalty is used, g_t includes its gradient [5; 12].

2.3.4. Overfitting and Regularization

Overfitting occurs when a model fits the training data more closely than is useful for predicting unseen examples. A widening training–validation performance gap can indicate overfitting. Regularization reduces this risk through constraints on learning or additional variability during training [1, Ch. 7].

$$\mathcal{J}_{\mathcal{B}}(\theta) = \mathcal{L}_{\mathcal{B}}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2$$

An L2 penalty discourages large values of the penalized parameters, contributing $\lambda\theta$ to their gradient. The project passes weight_decay to Adam, implementing this coupled penalty in the current training setup; this should not be confused with the decoupled update used by AdamW [12; 14].

Dropout randomly sets activations to zero during training to reduce reliance on particular units. With drop probability q, inverted dropout uses:

$$\tilde{a}_i = \frac{r_i a_i}{1-q}, \quad r_i \sim \text{Bernoulli}(1-q), \quad 0 \le q < 1$$

The scaling preserves the expected activation; dropout is disabled during inference. The project uses channel-wise Dropout2d in convolutional blocks and ordinary dropout in the classifier. Data augmentation and early stopping provide additional regularization, as discussed in Sections 2.6.2 and 2.4.3 [6; 12].

2.4. Model Evaluation Metrics

2.4.1. Multiclass Confusion Matrix

For M mutually exclusive classes, the confusion matrix $C \in \mathbb{N}_0^{M\times M}$ counts predictions by true class and predicted class. Rows correspond to true labels and columns to predicted labels [7]:

$$C_{i,j} = \sum_{n=1}^{N} \mathbf{1}[\, y_n = i, \hat{y}_n = j]$$

Here, N is the number of evaluated images. Diagonal entries count correct predictions; off-diagonal entries identify specific confusions. In the project, the fixed order is D00, D10, D20, and D40, giving the following four-class structure [12]:

$$C = \begin{bmatrix} C_{1,1} & C_{1,2} & C_{1,3} & C_{1,4} \\ C_{2,1} & C_{2,2} & C_{2,3} & C_{2,4} \\ C_{3,1} & C_{3,2} & C_{3,3} & C_{3,4} \\ C_{4,1} & C_{4,2} & C_{4,3} & C_{4,4} \end{bmatrix}$$

For example, $C_{1,2}$ counts true D00 images predicted as D10. The sum of row i is the support of class i. A row-normalized matrix divides each nonempty row by its support, showing how that true class is distributed across predicted classes. Counts should also be retained so that differences in class frequency remain visible [7].

2.4.2. Accuracy and Multiclass Precision Recall and F1 Score

Overall accuracy is the fraction of evaluated images classified correctly:

$$\text{Accuracy} = \frac{\sum_{k=1}^{M} C_{k,k}}{N}, \quad N = \sum_{i=1}^{M}\sum_{j=1}^{M} C_{i,j}$$

For each class k, a one-versus-rest view defines its true positives, false positives, false negatives, and support:

$$\mathrm{TP}_k = C_{k,k}, \quad \mathrm{FP}_k = \sum\nolimits_{i \neq k} C_{i,k}\,, \quad \mathrm{FN}_k = \sum\nolimits_{j \neq k} C_{k,j}\,, \quad s_k = \sum\nolimits_j C_{k,j}$$

$$P_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FP}_k}, \quad R_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FN}_k}, \quad F_{1,k} = \frac{2\mathrm{TP}_k}{2\mathrm{TP}_k + \mathrm{FP}_k + \mathrm{FN}_k}$$

Precision measures how often predictions of class k are correct. Recall measures how many actual examples of that class are recovered. F1 combines precision and recall through their harmonic mean. If a denominator is zero, the metric requires an explicit convention; the project assigns zero using zero_division=0 [7; 12].

Macro and support-weighted averages summarize per-class scores. For $Q_k \in \{P_k, R_k, F_{1,k}\}$:

$$Q_{\mathrm{macro}} = \frac{1}{M}\sum\nolimits_{k=1}^{M} Q_k\,, \quad Q_{\mathrm{weighted}} = \sum\nolimits_{k=1}^{M} \frac{s_k}{N} Q_k$$

Macro averaging gives equal weight to every class, while weighted averaging gives more influence to frequent classes. Macro-F1 is the mean of the class F1 scores, not generally the harmonic mean of macro-precision and macro-recall. Reporting per-class and macro scores helps expose weak minority-class performance that accuracy or weighted scores can conceal [7].

Micro averaging first pools the one-versus-rest counts:

$$P_{\mathrm{micro}} = \frac{\sum_k \mathrm{TP}_k}{\sum_k (\mathrm{TP}_k + \mathrm{FP}_k)}, \quad R_{\mathrm{micro}} = \frac{\sum_k \mathrm{TP}_k}{\sum_k (\mathrm{TP}_k + \mathrm{FN}_k)}$$

With exactly one true and one predicted class per image and all classes included, micro-precision, micro-recall, and micro-F1 equal accuracy. The project reports per-class, macro, and weighted scores alongside overall accuracy [7; 12].

2.4.3. Training and Validation Loss and Early Stopping

Training loss measures the objective on training examples, while validation loss measures predictive error on held-out examples without updating model parameters. The project records sample-weighted mean cross-entropy for each epoch. Its validation pass uses evaluation mode, so dropout is disabled and BN uses running statistics [12].

$$L_{\mathrm{val}}^{(t)} = -\frac{1}{N_{\mathrm{val}}}\sum\nolimits_{n=1}^{N_{\mathrm{val}}} \log p_{n,y_n}^{(t)}$$

A persistently decreasing training loss accompanied by a rising validation loss can indicate overfitting. Individual fluctuations are not conclusive, and training loss can exceed validation loss because training uses augmentation and dropout. Early stopping limits unnecessary training by monitoring whether validation loss improves sufficiently [1, Sec. 7.8].

The project uses two controls: min_delta is the minimum required reduction in validation loss, and patience is the number of consecutive epochs allowed without such an improvement. Let Δ denote min_delta, P denote patience, r_{t-1} be the last accepted reference loss, and q_t count consecutive non-improving epochs. Its rule is [12]:

$$\text{meaningful improvement at epoch} t \quad \Leftrightarrow \quad L_{\text{val}}^{(t)} < r_{t-1} - \Delta$$

$$(r_t, q_t) = \begin{cases} (L_{\text{val}}^{(t)}, 0), & L_{\text{val}}^{(t)} < r_{t-1} - \Delta, \\ (r_{t-1}, q_{t-1} + 1), & \text{otherwise.} \end{cases}$$

Initialize $r_0 = +\infty$ and $q_0 = 0$. Training stops when $q_t \geq P$ and $P > 0$; patience 0 disables this stopping rule. The current Complex CNN configuration uses patience 10 and min_delta 0.0001. For example, after accepting loss 0.5000, a new value must be below 0.4999 to reset the counter. Smaller changes are compared against the unchanged reference in subsequent epochs [12].

Stopping and checkpoint selection are separate. The current configuration saves the highest validation-accuracy checkpoint, using lower validation loss to break ties. Improvements smaller than min_delta can still update that checkpoint. Evaluation explicitly loads the saved checkpoint; early stopping itself does not restore the in-memory model. The learning-rate scheduler also monitors validation loss, but uses its own patience setting [12].

2.5. Representative CNN Architectures

2.5.1. LeNet

LeNet-5 is an early CNN designed for handwritten character recognition. It processes a 32×32 grayscale input through convolution, subsampling, and classification stages. Its C1 layer produces six feature maps, S2 reduces their spatial resolution, C3 produces 16 feature maps, and S4 performs further subsampling. C5 combines the remaining spatial features, followed by the F6 layer and the output stage [8].

The original architecture uses sigmoid-type nonlinearities and trainable subsampling, with a radial-basis output layer. Modern teaching implementations often replace these details with ReLU, ordinary pooling, and a softmax classifier. LeNet illustrates the central CNN pattern of extracting local features, reducing spatial resolution, and combining the resulting representations for classification [8].

2.5.2. VGG

VGG uses repeated 3×3 convolutions with stride 1 and padding that preserves spatial size within each block. ReLU follows the convolutions, while 2×2 max pooling with stride 2 reduces resolution between blocks. The number of feature channels increases as spatial resolution decreases. VGG-16 has 13 convolutional and three fully connected weight layers; VGG-19 has 16 convolutional and three fully connected weight layers [9].

Stacking small kernels expands the effective receptive field while introducing additional nonlinear transformations. VGG provides a regular architecture that is straightforward to describe and adapt, although its original fully connected classifier accounts for a large share of its parameters and memory requirements [9].

2.5.3. ResNet

ResNet introduces residual blocks to ease the optimization of deep networks. A block learns a residual transformation $F(\mathbf{x}; W)$ and adds a shortcut from its input [10]:

$$\mathbf{y} = F(\mathbf{x}; W) + \mathbf{x}$$

When the spatial dimensions or channel counts differ, a projection W_s, commonly implemented by a 1×1 convolution with a suitable stride, matches the shortcut to the residual output:

$$\mathbf{y} = F(\mathbf{x}; W) + W_s\mathbf{x}$$

The addition provides a direct path for information and gradients, allowing the block to learn a change to its input representation. ResNet-18 and ResNet-34 use basic blocks with two 3×3 convolutions; deeper variants such as ResNet-50 use bottleneck blocks with 1×1, 3×3, and 1×1 convolutions. The original networks finish with global average pooling and a fully connected classifier [10].

2.6. Techniques for Improving CNN Performance

2.6.1. Transfer Learning

Transfer learning builds a classifier from a CNN pretrained on a source dataset, such as ImageNet. The pretrained feature extractor is retained, and its original classification layer is replaced by a new head with one output per target class. For this four-class task, the new head therefore has four logits [11].

In a feature-extraction setup, the backbone weights are frozen and only the new head is trained. For fine-tuning, selected backbone layers or the full backbone are unfrozen and updated, usually with a smaller learning rate than the newly initialized head. Input preprocessing must match the pretrained weights, and the amount of fine-tuning is selected using validation performance. If the backbone is intended to remain fixed, its BN running statistics must also remain fixed [11].

2.6.2. Data Augmentation

Data augmentation generates varied training inputs through transformations intended to preserve class identity. Random flips, modest geometric changes, and photometric changes expose the model to variations that may occur in unseen images. Transformations must remain appropriate to the task; for example, large rotations can change the directional meaning of a road crack [1, Sec. 7.4].

The project applies online horizontal flipping, small affine transformations, color jitter, occasional perspective distortion, and Gaussian blur before tensor conversion and normalization. Augmentation is restricted to training images. Validation and test images receive only tensor conversion and normalization, using statistics computed from the training set. This preserves a consistent evaluation procedure and avoids using validation or test statistics to prepare the training inputs [12].

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016. Chapters 5–9.
https://www.deeplearningbook.org/

[2] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, Dive into Deep Learning. Cambridge University Press, 2023. Online edition, Sections 5.3 and 7.2–7.5.
https://d2l.ai/

[3] PyTorch contributors, “Conv2d,” PyTorch documentation. Accessed October 10, 2026.
https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html

[4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in Proceedings of the 32nd International Conference on Machine Learning, vol. 37, pp. 448–456, 2015. Algorithm 1 and Section 3.2.
https://proceedings.mlr.press/v37/ioffe15.html

[5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” International Conference on Learning Representations, 2015.
https://arxiv.org/abs/1412.6980

[6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” Journal of Machine Learning Research, vol. 15, no. 56, pp. 1929–1958, 2014.
https://jmlr.org/papers/v15/srivastava14a.html

[7] Scikit-learn developers, “Metrics and scoring: quantifying the quality of predictions,” Scikit-learn User Guide. Accessed October 10, 2026.
https://scikit-learn.org/stable/modules/model_evaluation.html

[8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791.
https://bottou.org/papers/lecun-98h

[9] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” International Conference on Learning Representations, 2015.
https://arxiv.org/abs/1409.1556

[10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778, 2016.
https://arxiv.org/abs/1512.03385

[11] Stanford University, “Transfer Learning,” CS231n course notes. Accessed October 10, 2026.
https://cs231n.github.io/transfer-learning/

[12] Road damage classification project, local source code and configuration: train.py, evaluate.py, dataset.py, models/complex_cnn.py, and configs/complex_cnn.json. Implementation inspected October 10, 2026.

[13] PyTorch contributors, “CrossEntropyLoss,” PyTorch documentation. Accessed October 10, 2026.
https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

[14] PyTorch contributors, “Adam,” PyTorch documentation. Accessed October 10, 2026.
https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html